

FIT 3080: Artificial Intelligence

Markov Decision Processes (MDPs)

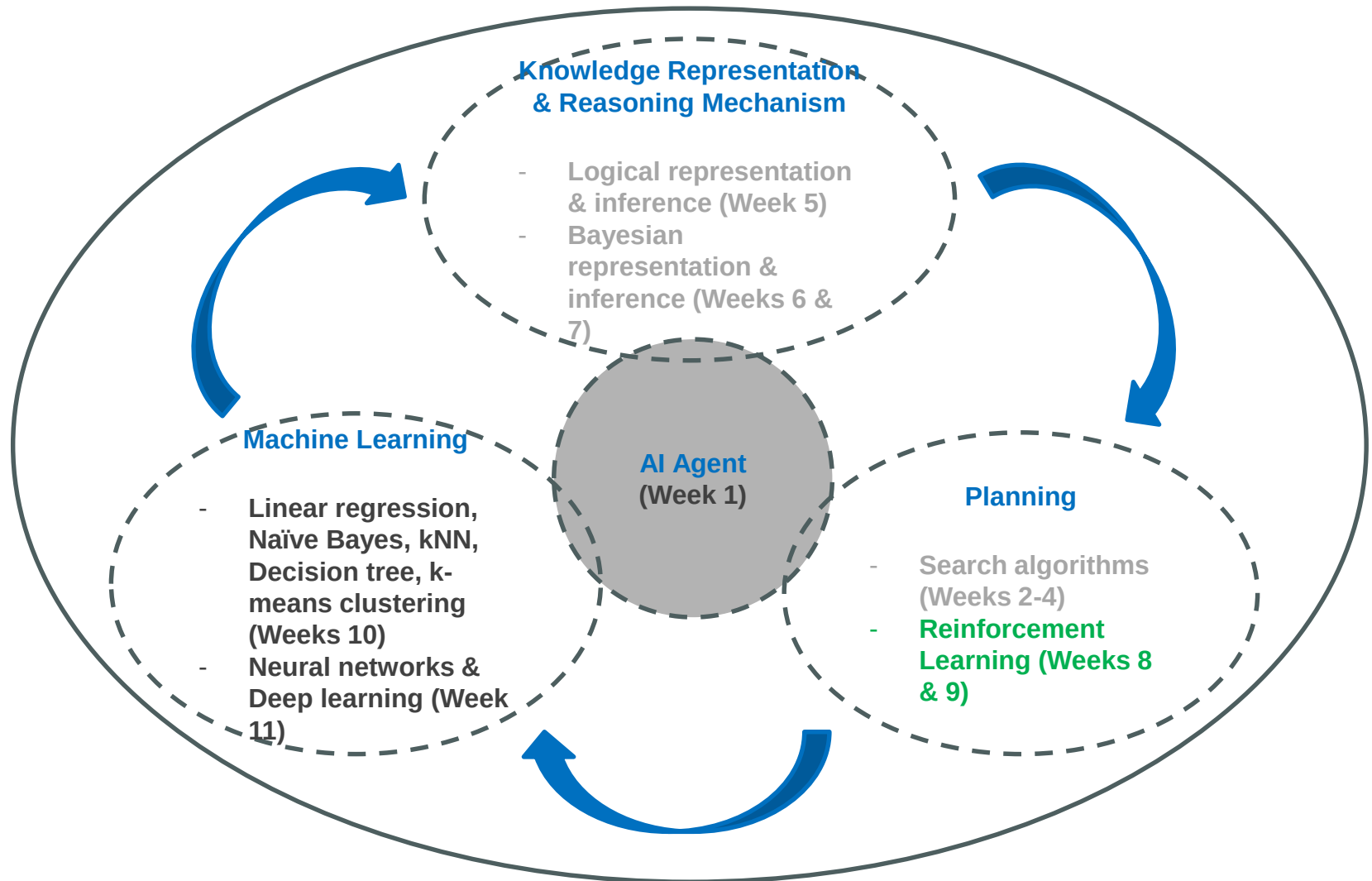
Presented by Jianfei Cai
Faculty of Information Technology
Monash University

Slides over the course partially adapted from

Stuart Russell, Andrew Moore, or Dan Klein

CS 287: Advanced Robotics (Fall 2019), Pieter Abbeel

Overview of Unit topics



Environment: Search, MDP & RL

	Search Algorithms	Model-based MDP (Bellman)	Reinforcement Learning
Environment Assumptions	fully observable	fully observable	partially observable
	known	known	initially unknown
	deterministic	stochastic	stochastic

Non-Deterministic Search

- How do you search for a plan when your actions might fail?



Outline

- MDPs
 - Definition & History
 - MDPs as Search Trees
 - Utility of a Sequence of Actions
- Solving MDPs
 - Bellman Equation
 - Value Iteration
 - Policy Iteration
- Partial Observable MDP (POMDP)

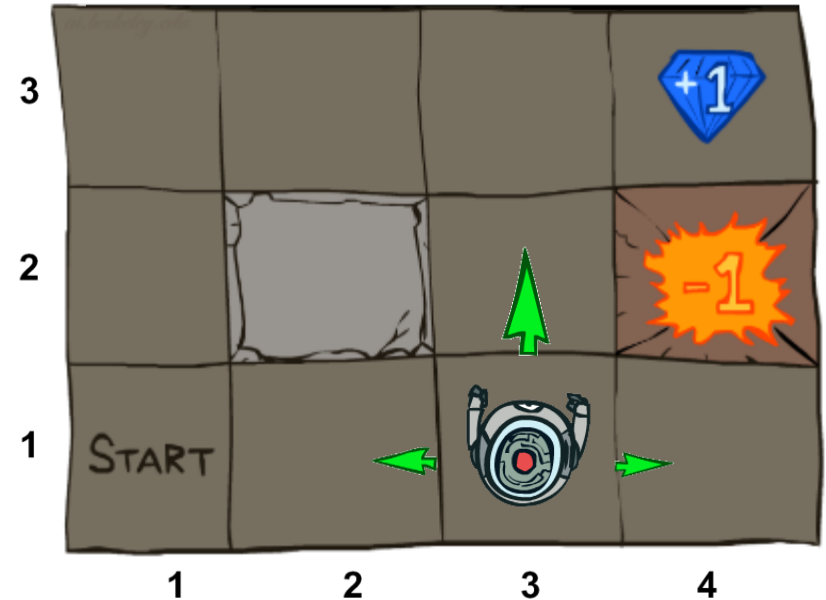
Outline

- MDPs
 - Definition & History
 - MDPs as Search Trees
 - Utility of a Sequence of Actions
- Solving MDPs
 - Bellman Equation
 - Value Iteration
 - Policy Iteration
- Partial Observable MDP (POMDP)

MDPs

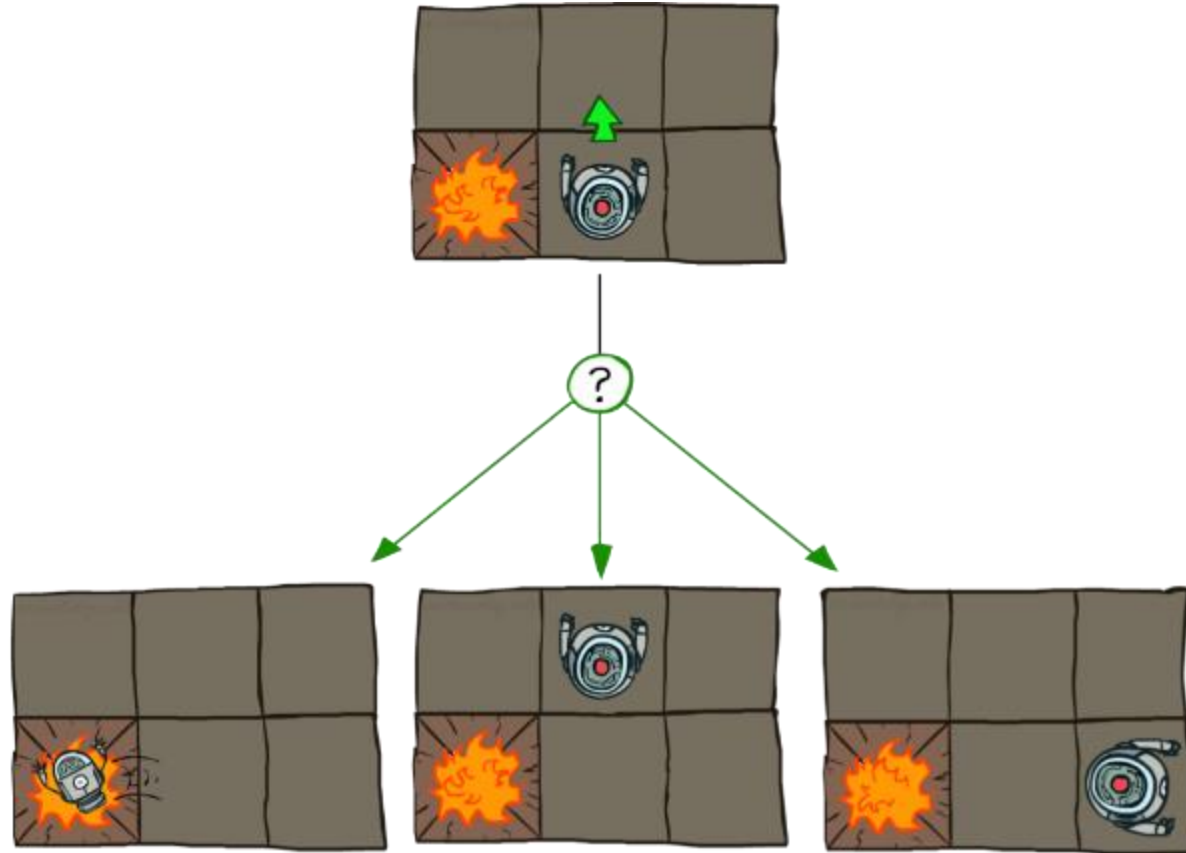
Example: Grid world

- Maze environment
- Noisy movement
 - 80% success,
 - 10% slip either side
- Rewards and penalties
 - Small reward for living
 - Big reward for diamond
- Goal: maximize sum of rewards



Robot example from UC Berkeley CS188:
Dan Klein and Pieter Abbeel

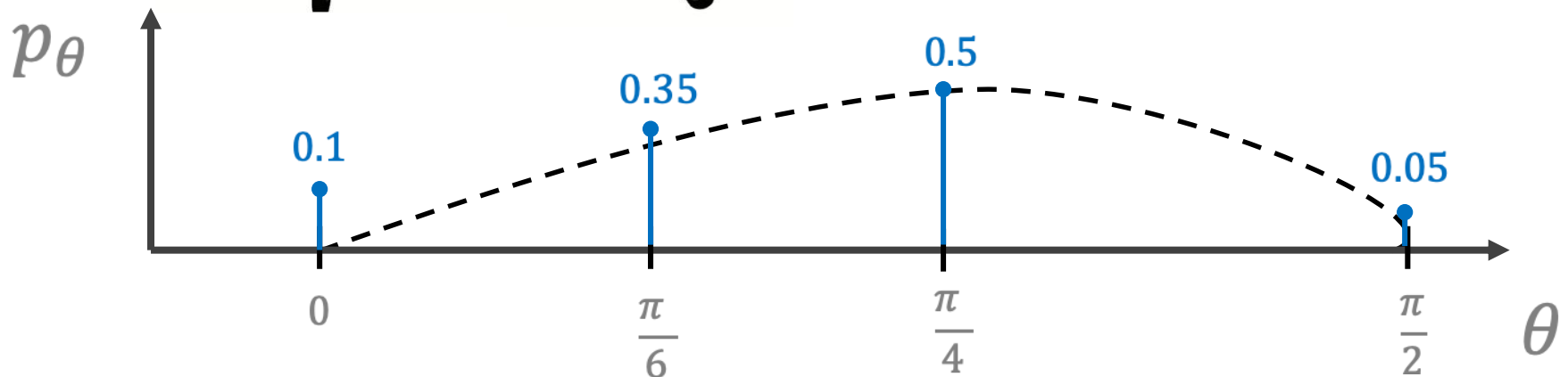
Actions and consequences



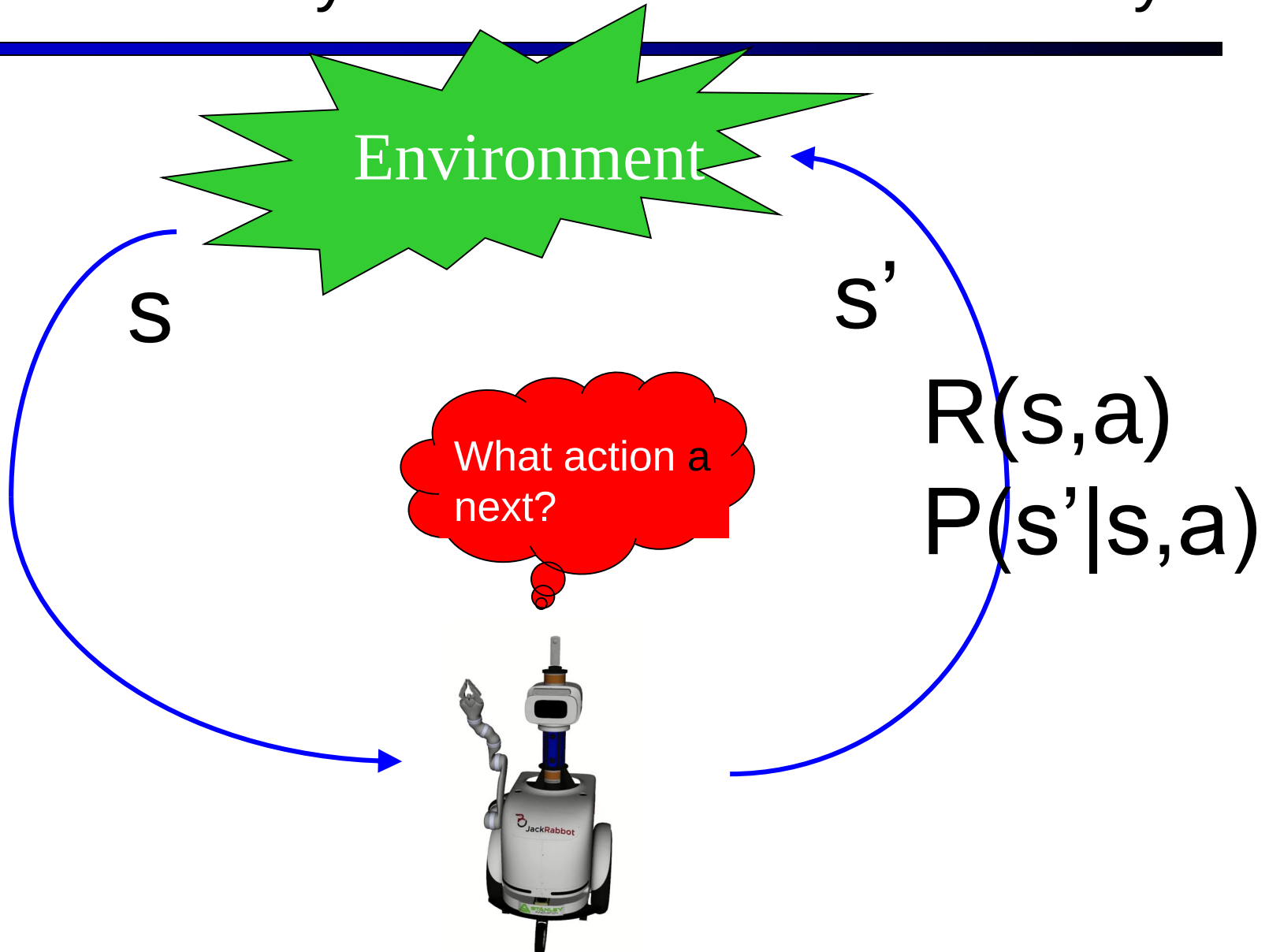
Robot example from UC Berkeley CS188:
Dan Klein and Pieter Abbeel

Real-World Robotics

- Example: turn 45 degree to the left

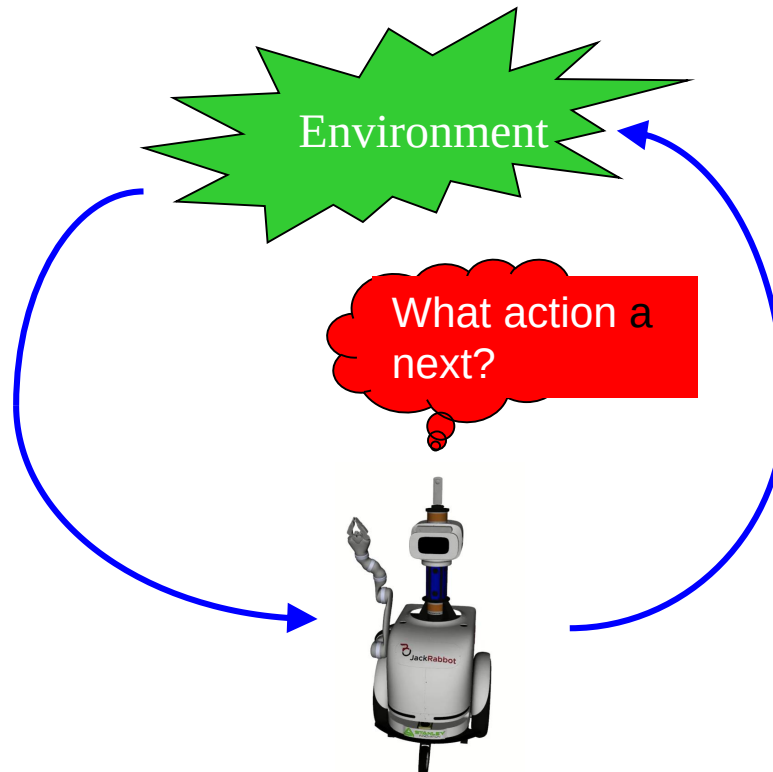


Robot's Life-Cycle: Plan Under Uncertainty



Robot's Life-Cycle: Plan Under Uncertainty

s_0 a_0 $r_0,$ s_1 a_1 $r_1,$ s_2 $\dots$



What is Markov about MDPs?

- Andrey Markov (1856-1922)
 - a Russian mathematician best known for his work on stochastic processes.
 - A primary subject of his research later became known as the [Markov chain](#).

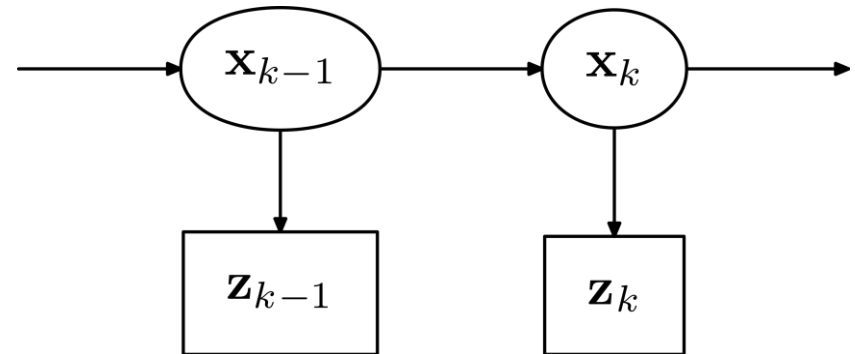


What is Markov about MDPs?

- Andrey Markov (1856-1922)
- “Markov” generally means that given the present state, the future and the past are independent

$$p(\mathbf{x}_k | \mathbf{x}_{k-1}, \mathbf{x}_{k-2}, \dots, \mathbf{x}_0) = p(\mathbf{x}_k | \mathbf{x}_{k-1})$$

$$p(\mathbf{z}_k | \mathbf{x}_k, \mathbf{x}_{k-1}, \dots, \mathbf{x}_0) = p(\mathbf{z}_k | \mathbf{x}_k)$$



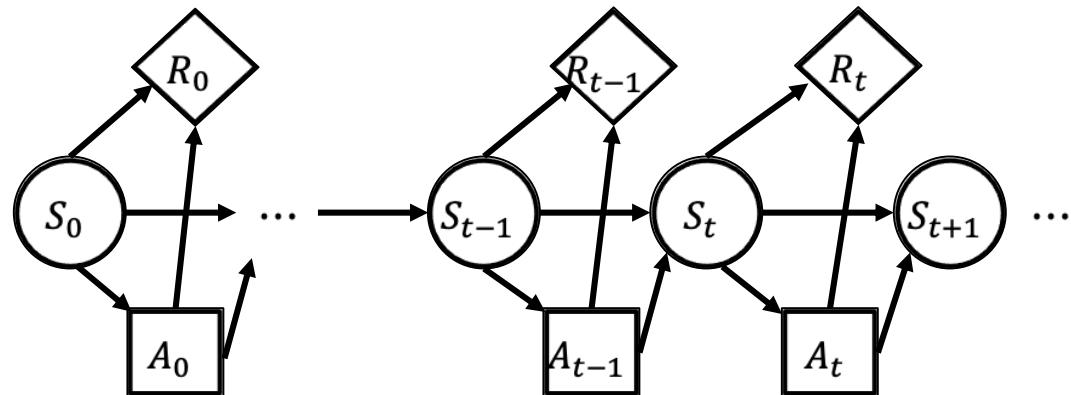
https://en.wikipedia.org/wiki/Recursive_Bayesian_estimation

$\mathbf{x}$ is assumed to be an unobserved Markov process, and the measurement $\mathbf{z}$ are the observation of a Hidden Markov model (HMM)

What is Markov about MDPs?

- Andrey Markov (1856-1922)
- “Markov” generally means that given the present state, the future and the past are independent
- For Markov decision processes, “Markov” means:

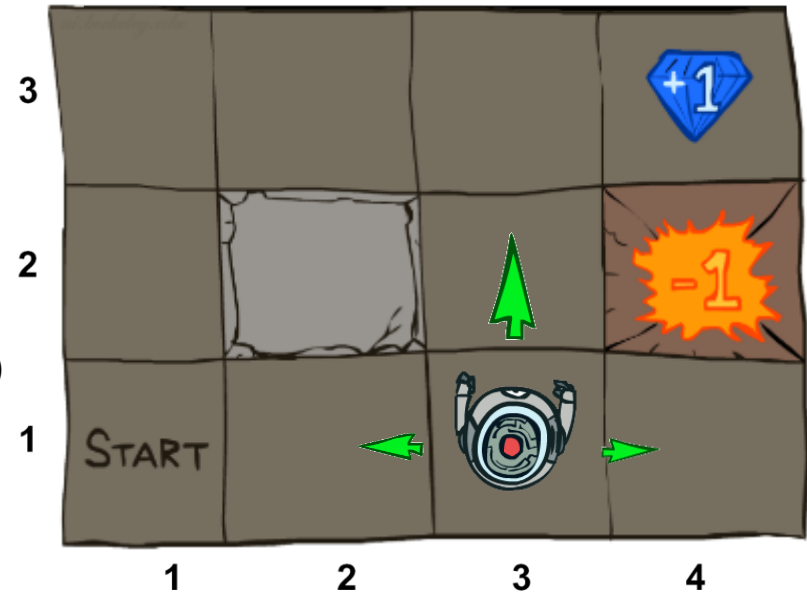
S = State,
 A = Action,
 R = Reward
 t = Time / sequence index



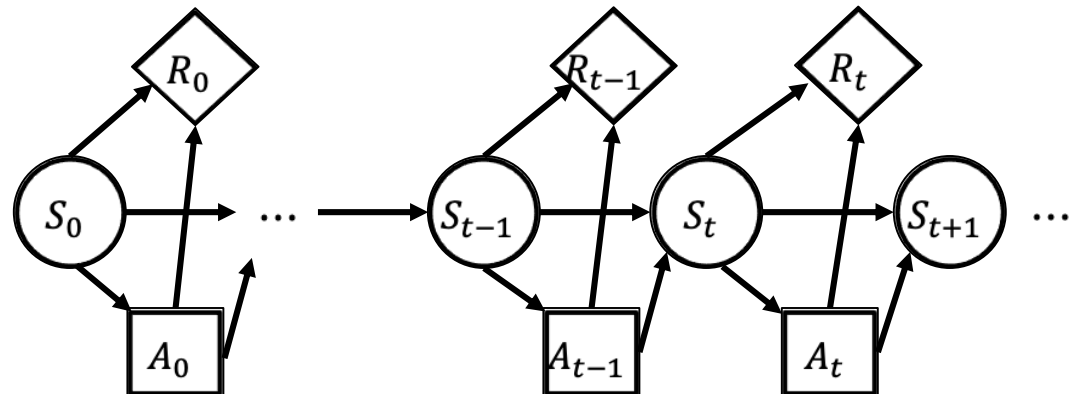
$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \dots, S_0 = s_0) = P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$

Markov Decision Processes

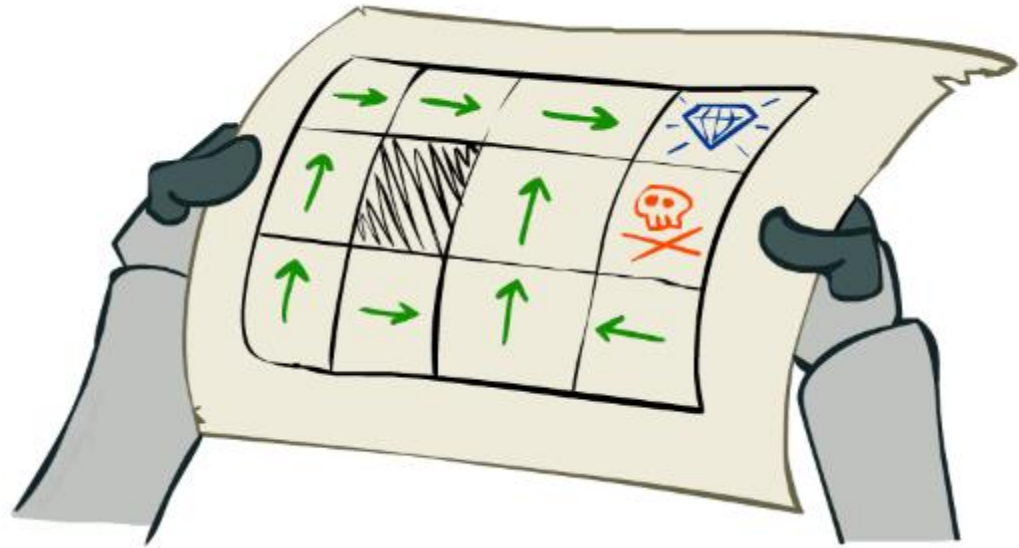
- An MDP is defined by
 - Set of states S
 - Set of actions A
 - Transition function $T(s,a,s')$
 - Gives the probability of s' , $P(s' | s, a)$
 - Reward function $R(s,a)$
 - Gives the value the agent receives



S = State,
 A = Action,
 R = Reward
 t = Time / sequence index



- Policy gives an action for each state:
- Optimal policy maximizes *expected* utility



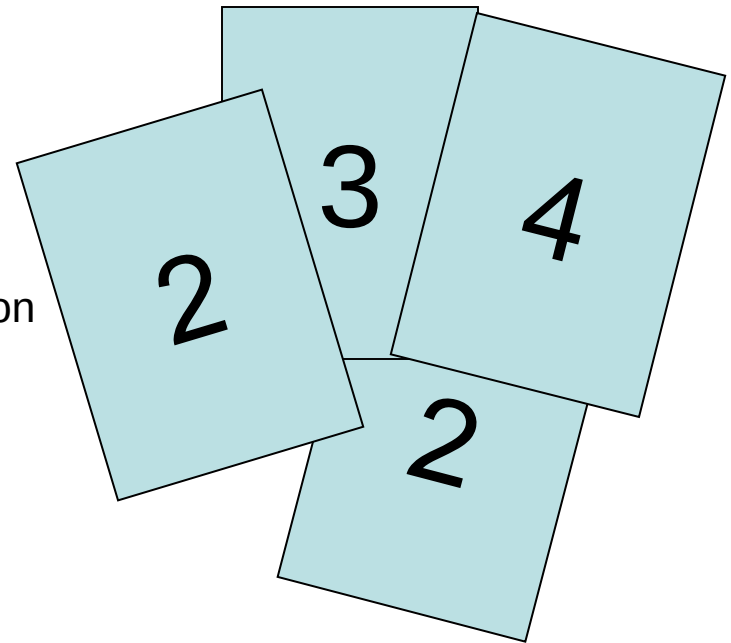
Outline

- MDPs
 - Definition & History
 - MDPs as Search Trees
 - Utility of a Sequence of Actions
- Solving MDPs
 - Bellman Equation
 - Value Iteration
 - Policy Iteration
- Partial Observable MDP (POMDP)

Example: High-Low

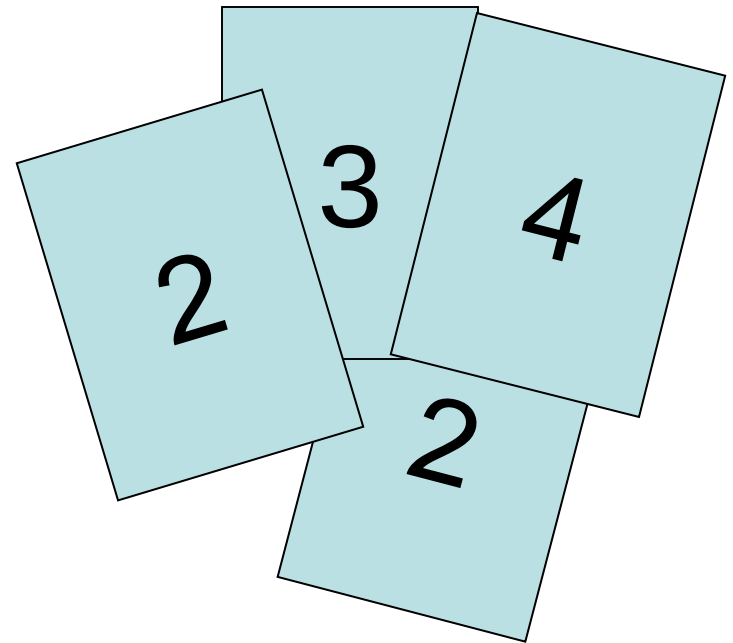
- Rules:

- Three card types: 2, 3, 4
 - Infinite deck, **twice as many 2's**
- Start with 3 showing
- After each card, you say “high” or “low”
 - New card is flipped
 - If you're right, you win the points shown on the new card
 - Ties are no-ops
 - If you're wrong, game ends



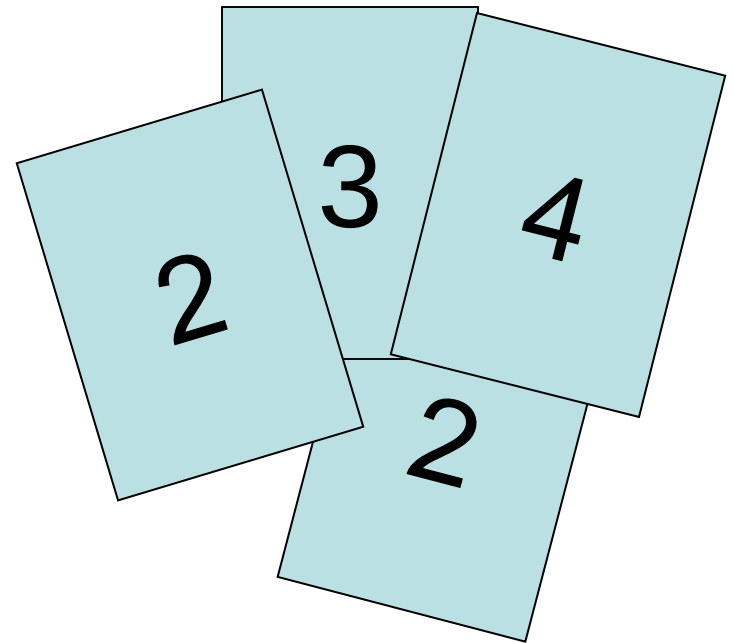
High-Low as an MDP

- States: 2, 3, 4, done
- Actions: High, Low
- Model: $T(s, a, s')$:
 - $P(s' = 4 \mid 4, \text{Low}) = 1/4$
 - $P(s' = 3 \mid 4, \text{Low}) = 1/4$
 - $P(s' = 2 \mid 4, \text{Low}) = 1/2$
 - $P(s' = \text{done} \mid 4, \text{Low}) = 0$

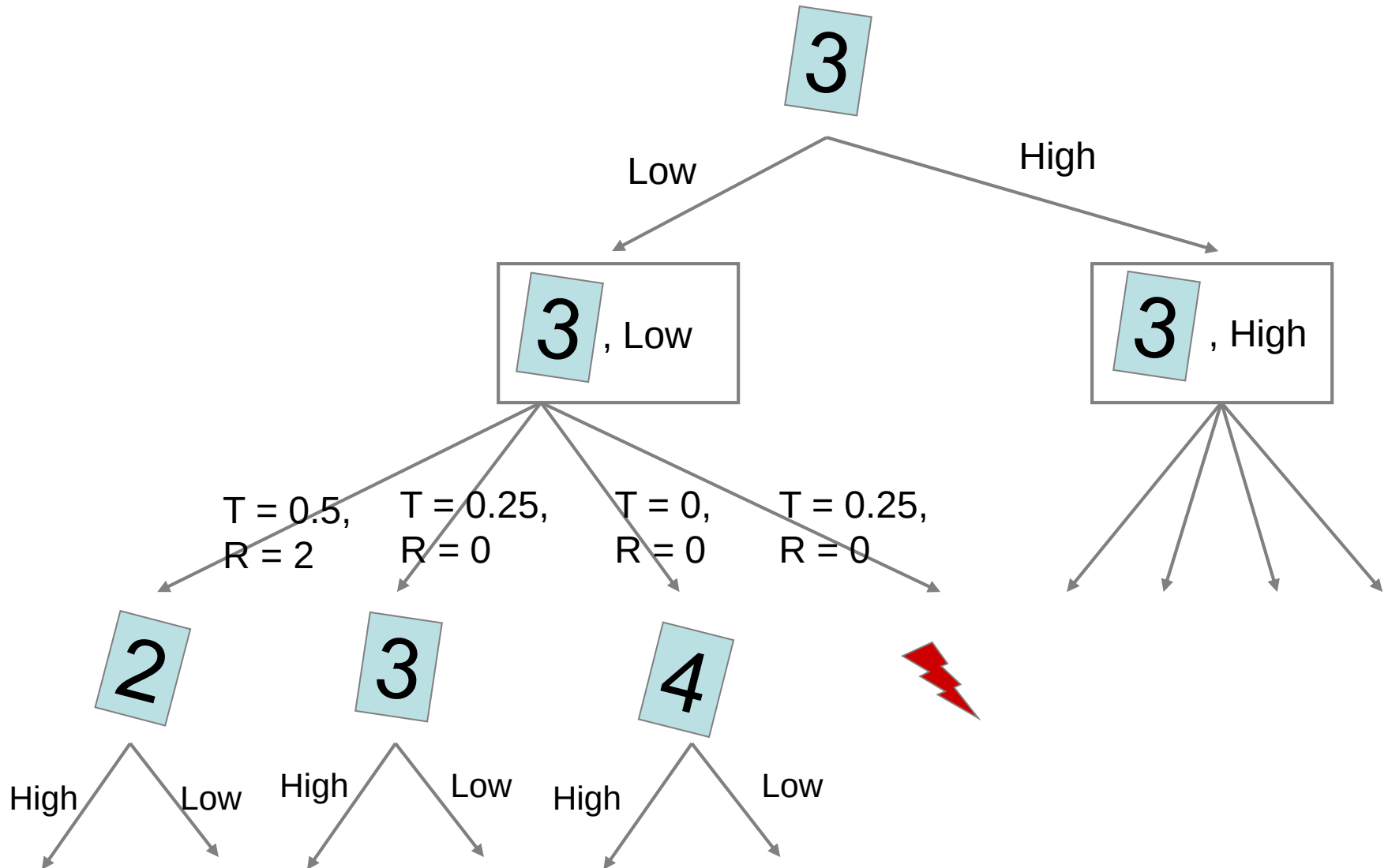


High-Low as an MDP

- States: 2, 3, 4, done
- Actions: High, Low
- Model: $T(s, a, s')$:
 - $P(s' = 4 \mid 4, \text{Low}) = 1/4$
 - $P(s' = 3 \mid 4, \text{Low}) = 1/4$
 - $P(s' = 2 \mid 4, \text{Low}) = 1/2$
 - $P(s' = \text{done} \mid 4, \text{Low}) = 0$
 - $P(s' = 4 \mid 4, \text{High}) = ?$
 - $P(s' = 3 \mid 4, \text{High}) = ?$
 - $P(s' = 2 \mid 4, \text{High}) = ?$
 - $P(s' = \text{done} \mid 4, \text{High}) = ?$
 - ...
- Rewards: $R(s, a, s')$:
 - Number shown on s' if $s \neq s'$
 - 0 otherwise

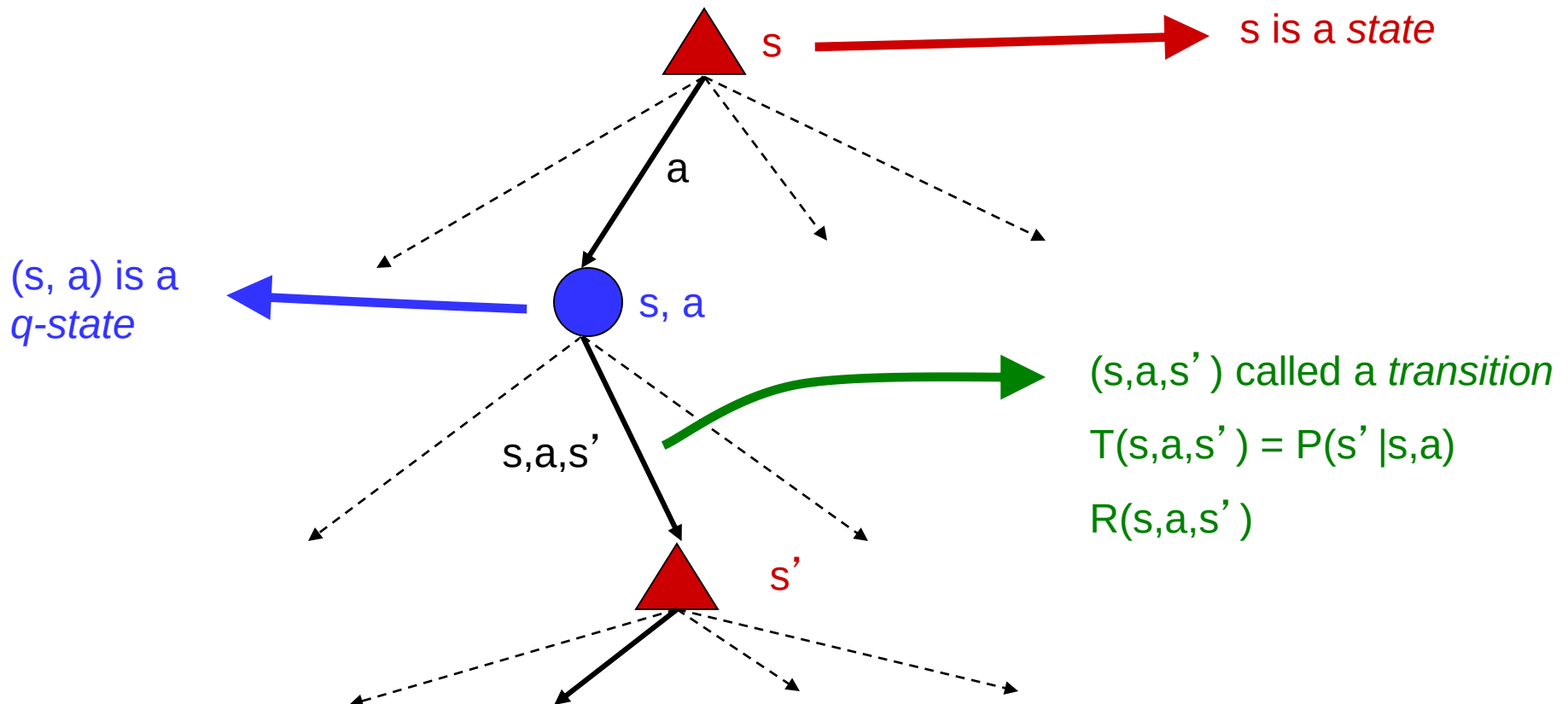


Example: High-Low



MDP Search Trees

- Each MDP state gives an “expectimax” search tree



Outline

- MDPs
 - Definition & History
 - MDPs as Search Trees
 - Utility of a Sequence of Actions
- Solving MDPs
 - Bellman Equation
 - Value Iteration
 - Policy Iteration
- Partial Observable MDP (POMDP)

Utilities of Sequences

- In order to formalize optimality of a policy, need to understand utilities of sequences of rewards
- Typically consider **stationary preferences**:

$$\begin{aligned} [r, r_0, r_1, r_2, \dots] &\succ [r, r'_0, r'_1, r'_2, \dots] \\ &\Leftrightarrow \\ [r_0, r_1, r_2, \dots] &\succ [r'_0, r'_1, r'_2, \dots] \end{aligned}$$

- **Theorem: only two ways to define stationary utilities**
 - Additive utility:

$$U([r_0, r_1, r_2, \dots]) = r_0 + r_1 + r_2 + \dots$$

- Discounted utility:

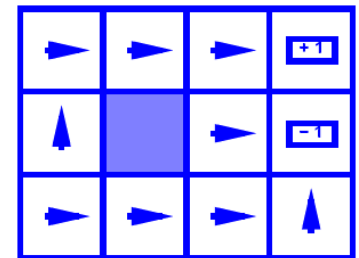
$$U([r_0, r_1, r_2, \dots]) = r_0 + \gamma r_1 + \gamma^2 r_2 \dots$$

Infinite Utilities?!

- Problem: infinite state sequences have infinite rewards

- Solutions:

- Finite horizon:
 - Terminate episodes after a fixed T steps (e.g. life)
 - Gives nonstationary policies (π depends on time left)
- Absorbing state: guarantee that for every policy, a terminal state will eventually be reached (like “done” for High-Low)
- Discounting: for $0 < \gamma < 1$

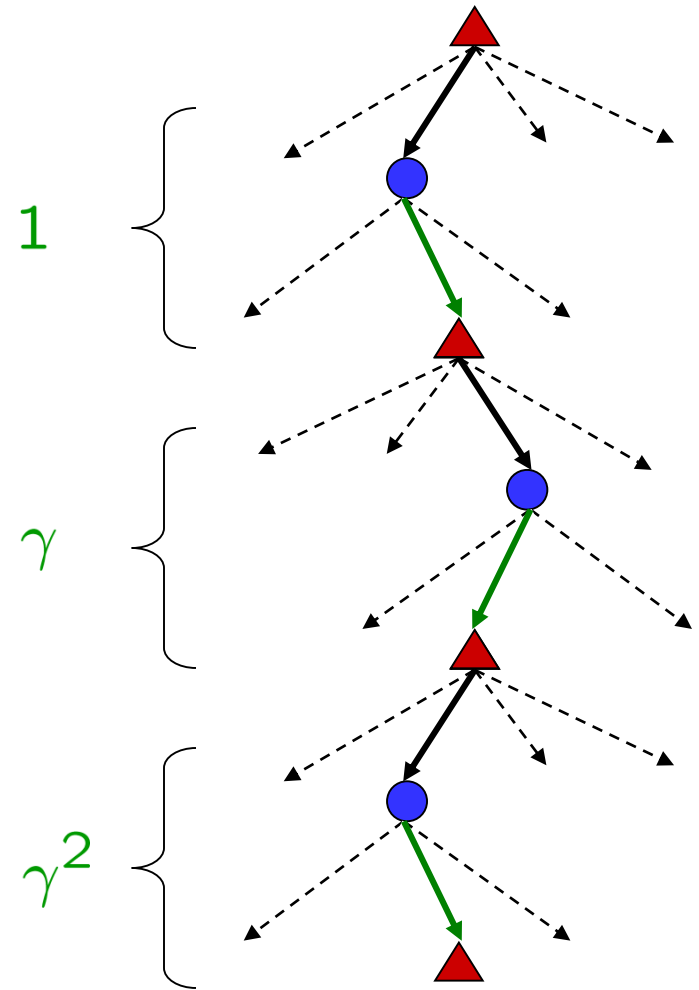


$$U([r_0, \dots, r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq R_{\max} / (1 - \gamma)$$

- Smaller γ means smaller “horizon” – shorter term focus

Discounting

- Typically discount rewards by $\gamma < 1$ each time step
 - Sooner rewards have higher utility than later rewards
 - Also helps the algorithms converge

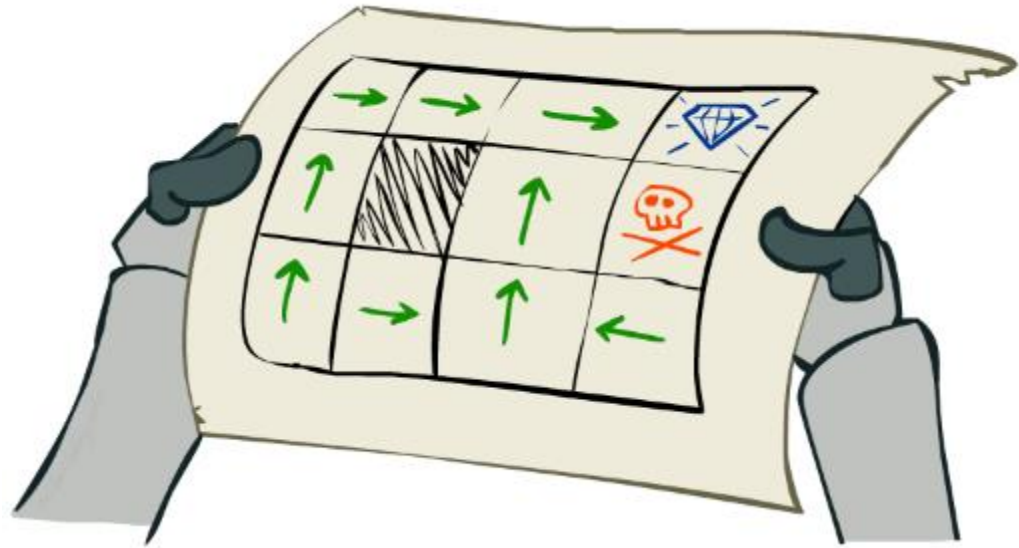


Outline

- MDPs
 - Definition & History
 - MDPs as Search Trees
 - Utility of a Sequence of Actions
- Solving MDPs
 - Bellman Equation
 - Value Iteration
 - Policy Iteration
- Partial Observable MDP (POMDP)

Solving MDPs

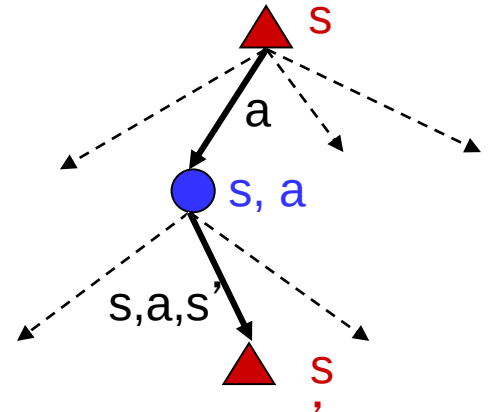
- Policy gives an action for each state:
- Optimal policy maximizes *expected* utility



Defining MDPs

- Markov decision processes:

- States S
- Start state s_0
- Actions A
- Transitions $P(s' | s, a)$ (or $T(s, a, s')$)
- Rewards $R(s, a, s')$ (and discount γ)



- MDP quantities so far:

- Policy = Choice of action for each state
- Utility (or return) = sum of discounted rewards

Solving MDPs

- In deterministic single-agent search problems, want an optimal **plan**, or sequence of actions, from start to a goal

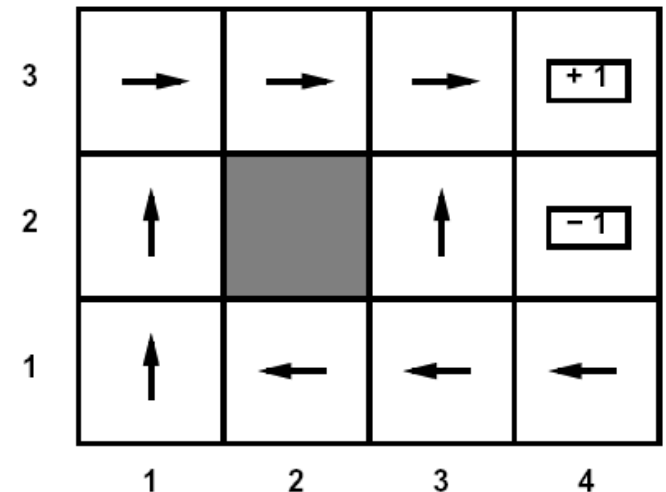
- In an MDP, we want an optimal **policy** $\pi^*: S \rightarrow A$

- A policy π gives an action for each state
- An optimal policy maximizes **expected utility** if followed

$$U([r_0, r_1, r_2, \dots]) = r_0 + \gamma r_1 + \gamma^2 r_2 \dots$$

- Defines a reflex agent

Optimal policy when
 $R(s, a) = -0.03$ for all
non-terminals s



Planning a policy

- Solution to an MDP is a policy, $\pi^*: S \rightarrow A$
 - Policy gives an action for each state:
 - Optimal policy maximizes *expected* utility
- Maximum expected utility is recursive, **Bellman equation**:

$$V^*(s) = \max_{a \in A} \left(R(s, a) + \sum_{s' \in S} (T(s, a, s') \cdot V^*(s')) \right)$$

Value = best action

Planning a policy

- Solution to an MDP is a policy, $\pi^*: S \rightarrow A$
 - Policy gives an action for each state:
 - Optimal policy maximizes *expected* utility
- Maximum expected utility is recursive, **Bellman equation:**

$$V^*(s) = \max_{a \in A} \left(R(s, a) + \sum_{s' \in S} (T(s, a, s') \cdot V^*(s')) \right)$$

Best action = current + future expected reward

Optimal Utilities

- Define the value of a state s :

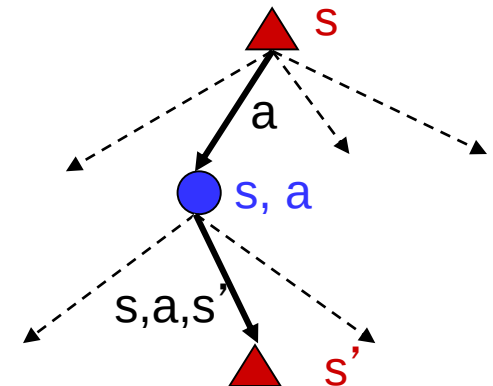
$V^*(s)$ = expected utility starting in s and acting optimally

- Define the value of a q-state (s,a) :

$Q^*(s,a)$ = expected utility starting in s , taking action a and thereafter acting optimally

- Define the optimal policy:

$\pi^*(s)$ = optimal action from state s



3	0.812	0.868	0.912	$\boxed{+1}$
2	0.762		0.660	$\boxed{-1}$
1	0.705	0.655	0.611	0.388
	1	2	3	4

3	→	→	→	$\boxed{+1}$
2	↑		↑	$\boxed{-1}$
1	↑	←	←	←
	1	2	3	4

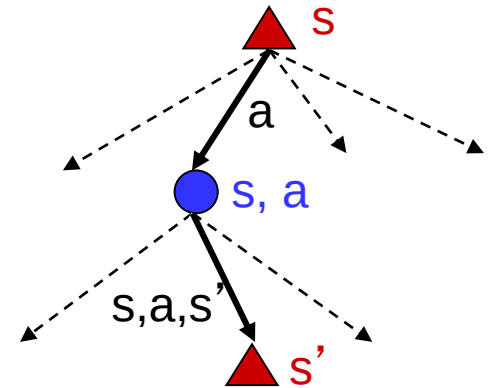
Solving MDPs

- We want to find the optimal policy π^*
- How to do that?

The Bellman Equations

- Definition of “optimal utility” leads to a simple one-step lookahead relationship amongst optimal utility values:

Optimal rewards = maximize over first action and then follow optimal policy



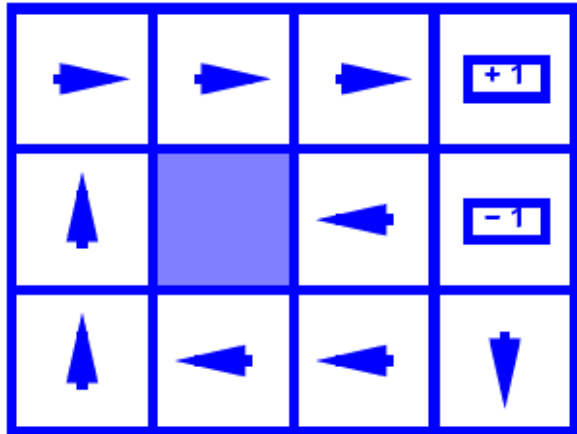
- Formally:

$$V^*(s) = \max_a Q^*(s, a)$$

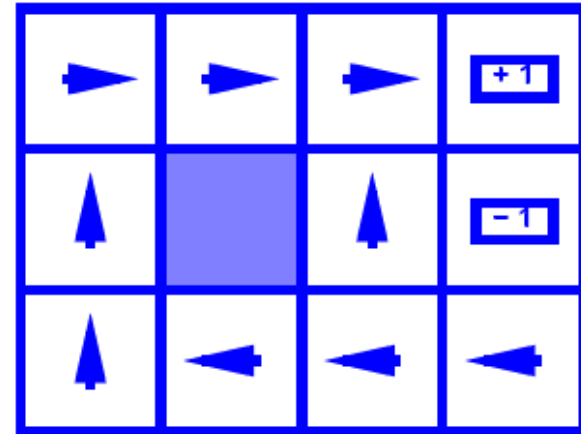
$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

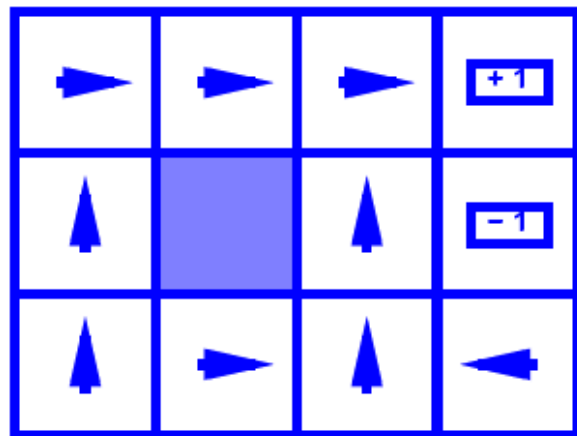
Example Optimal Policies



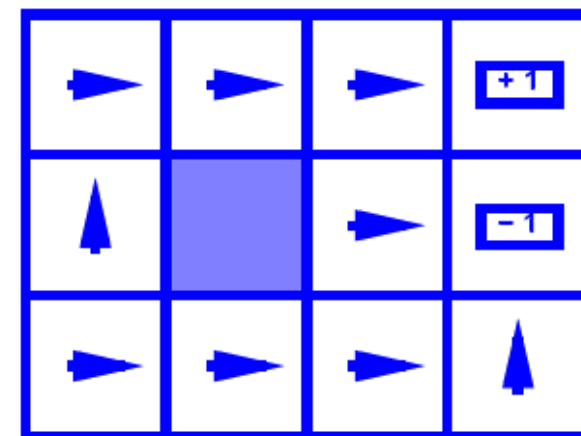
$$R(s) = -0.01$$



$$R(s) = -0.03$$



$$R(s) = -0.4$$



$$R(s) = -2.0$$

Outline

- MDPs
 - Definition & History
 - MDPs as Search Trees
 - Utility of a Sequence of Actions
- Solving MDPs
 - Bellman Equation
 - Value Iteration
 - Policy Iteration
- Partial Observable MDP (POMDP)

Value Iteration

- Idea:

- Start with $V_0^*(s) = 0$.
- Given V_i^* , calculate the values for all states for depth $i+1$:

$$V_{i+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_i(s')]$$

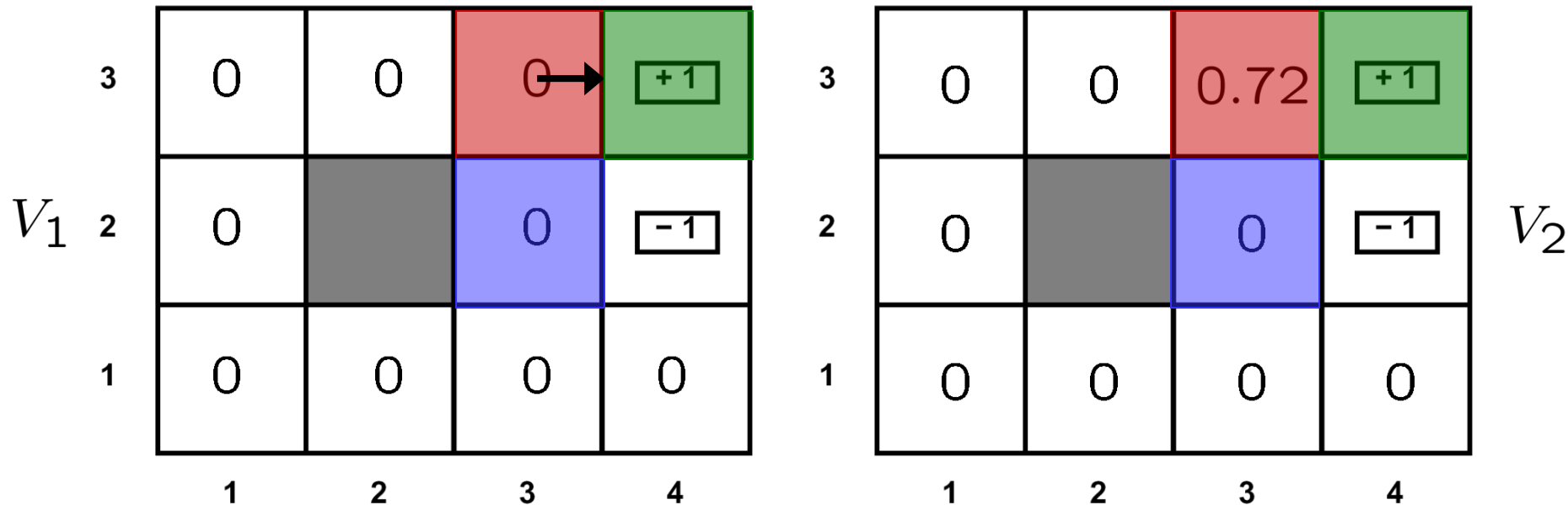
- This is called a **value update** or **Bellman update**
 - Repeat until convergence
- Theorem: will converge to unique optimal values
 - Basic idea: approximations get refined towards optimal values
 - Policy may converge long before values do

Example: Value Iteration

noise = 0.2, $\gamma = 0.9$, two terminal states with $R = +1$ and -1 & values = 0



Example: Bellman Updates



$$V_{i+1}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_i(s')]$$

$$V_2(\langle 3, 3 \rangle) = \sum_{s'} T(\langle 3, 3 \rangle, \text{right}, s') [R(\langle 3, 3 \rangle) + 0.9 V_1(s')]$$

max happens for
 $a=\text{right}$, other
actions not shown

$$= 0.9 [0.8 \cdot 1 + 0.1 \cdot 0 + 0.1 \cdot 0]$$

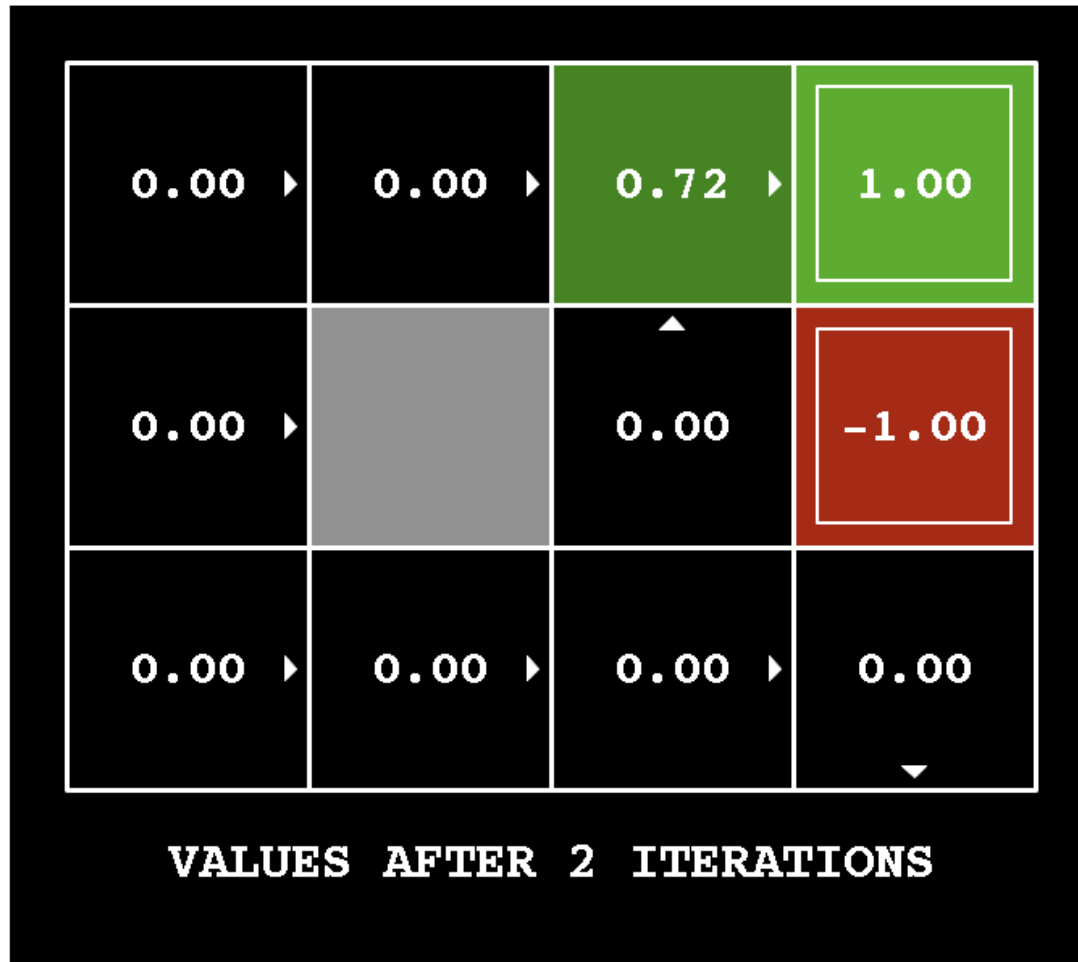
Example: Value Iteration

noise = 0.2, $\gamma = 0.9$, two terminal states with $R = +1$ and -1 & values = 0



Example: Value Iteration

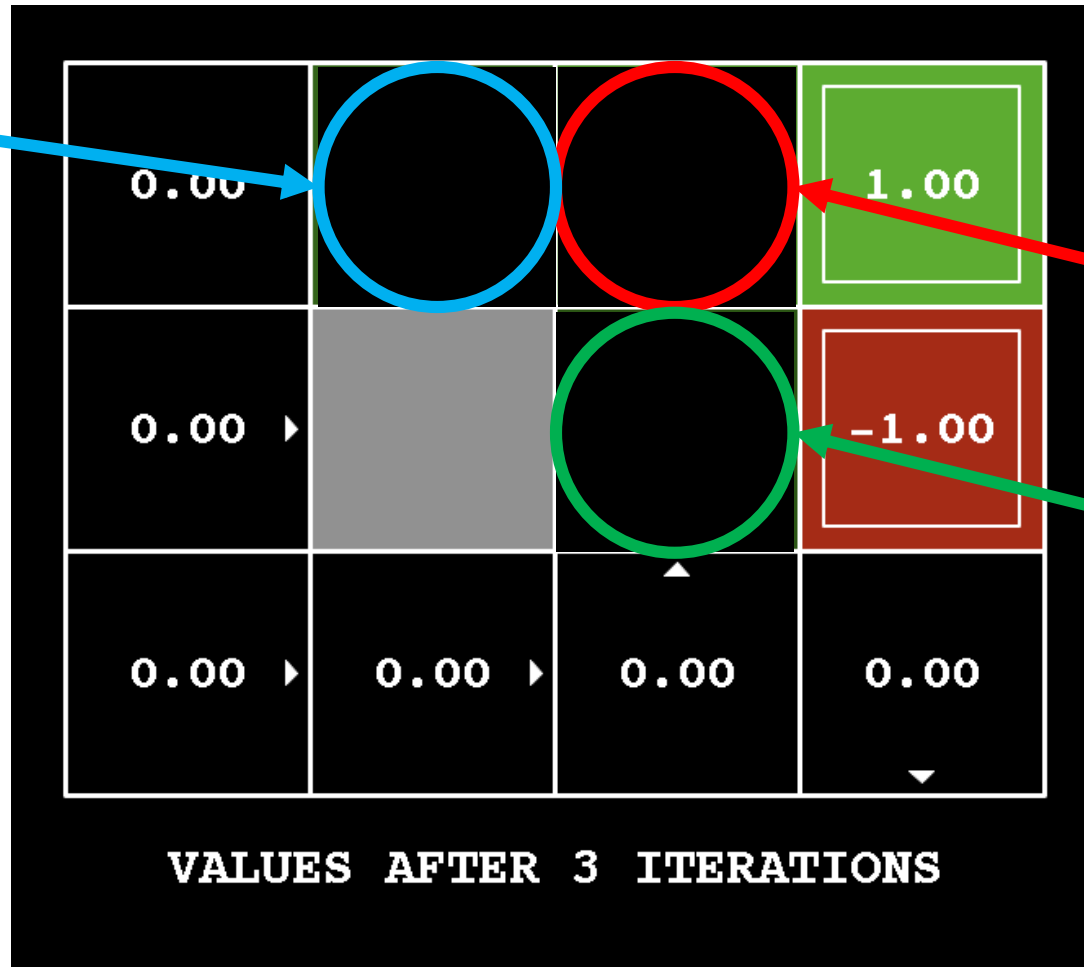
noise = 0.2, $\gamma = 0.9$, two terminal states with $R = +1$ and -1 & values = 0



Example: Value Iteration

noise = 0.2, $\gamma = 0.9$, two terminal states with $R = +1$ and -1 & values = 0

Blue Circle

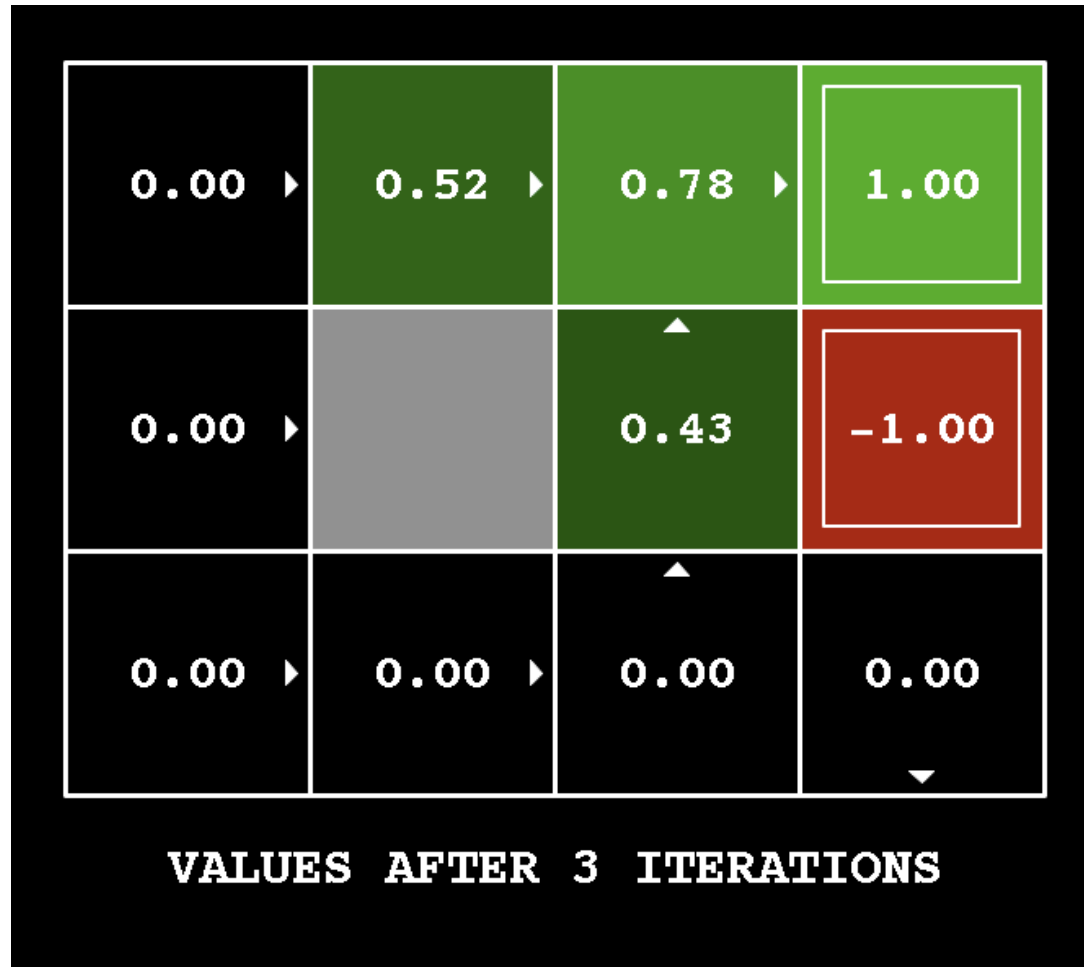


Red Circle

Green Circle

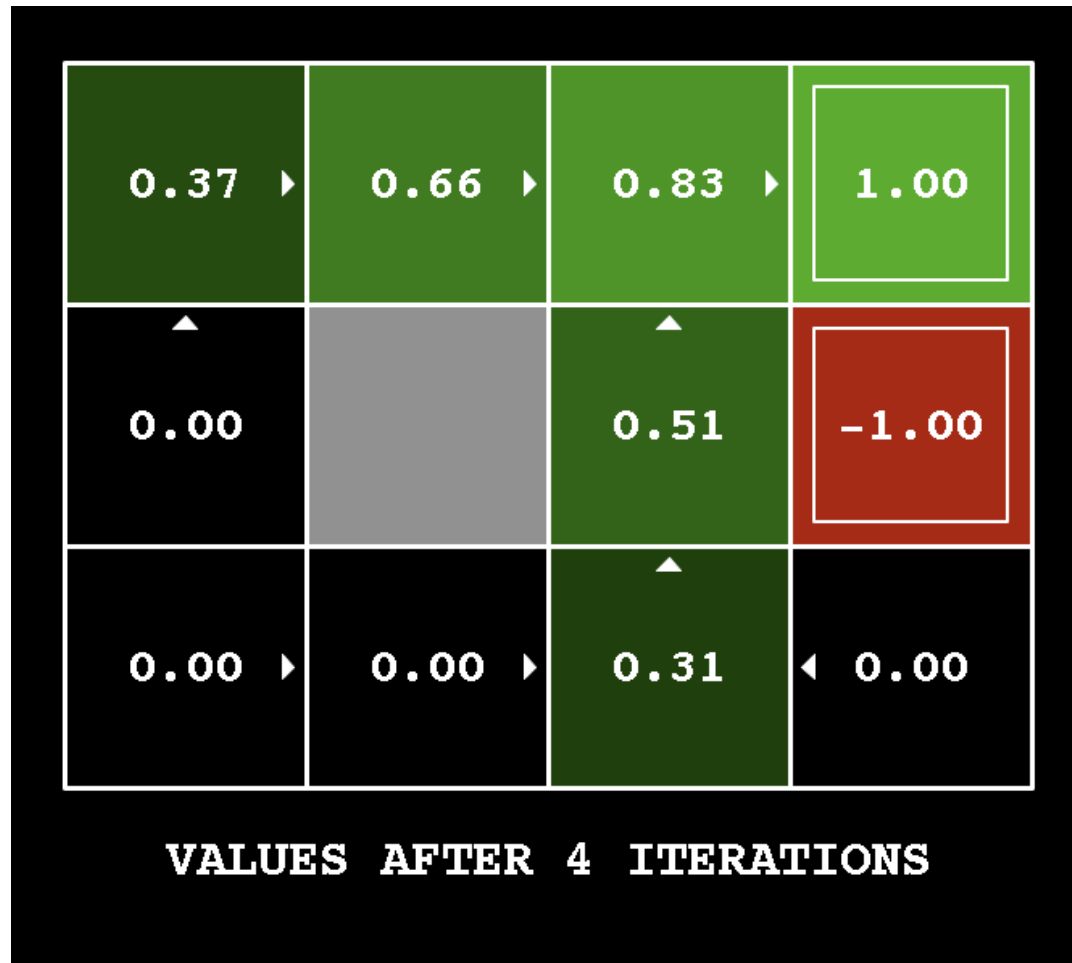
Example: Value Iteration

noise = 0.2, $\gamma = 0.9$, two terminal states with $R = +1$ and -1 & values = 0



Example: Value Iteration

noise = 0.2, $\gamma = 0.9$, two terminal states with $R = +1$ and -1 & values = 0



Example: Value Iteration

noise = 0.2, $\gamma = 0.9$, two terminal states with $R = +1$ and -1



Example: Value Iteration

noise = 0.2, $\gamma = 0.9$, two terminal states with $R = +1$ and -1

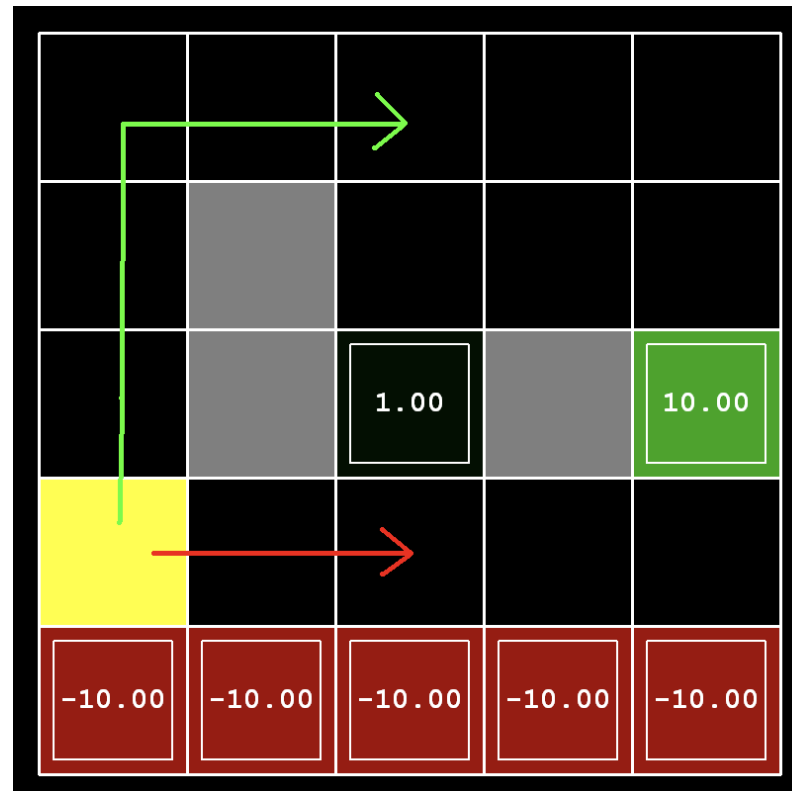


Example: Value Iteration

noise = 0.2, $\gamma = 0.9$, two terminal states with $R = +1$ and -1



Effect of Discount and Noise



- a) Prefer the close exit (+1), risking the cliff (-10)
- b) Prefer the close exit (+1), avoiding the cliff (-10)
- c) Prefer the distant exit (+10), risking the cliff (-10)
- d) Prefer the distant exit (+10), avoiding the cliff (-10)

- (1) $\gamma = 0.1$, noise = 0.5
- (2) $\gamma = 0.99$, noise = 0
- (3) $\gamma = 0.99$, noise = 0.5
- (4) $\gamma = 0.1$, noise = 0

Effect of Discount and Noise

0.00 ▸	0.00 ▸	0.01 ▼	0.01 ▸	0.10 ▼
0.00 ▼		0.10 ▼	0.10 ▸	1.00 ▼
0.00 ▼		1.00		10.00
0.00 ▸	0.01 ▸	0.10 ▲	0.10 ▸	1.00 ▲
-10.00	-10.00	-10.00	-10.00	-10.00

a) Prefer the close exit (+1), risking the cliff (-10)

(4) $\gamma = 0.1$, noise = 0

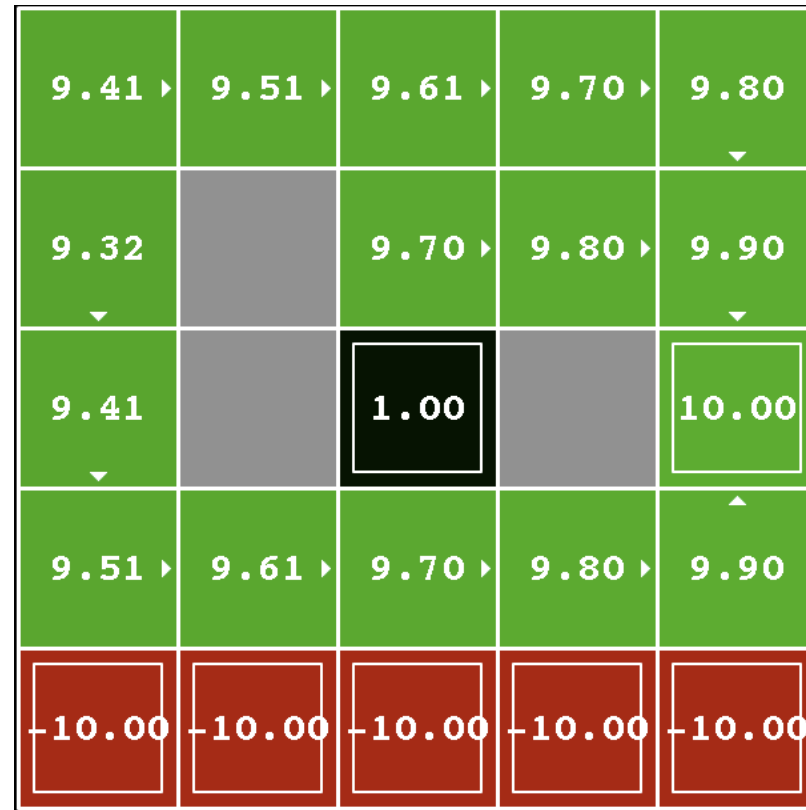
Effect of Discount and Noise

0.00 ▸	0.00 ▸	0.00 ▼	0.00 ▼	0.03 ▼
▲ 0.00		0.05 ▼	0.03 ▸	0.51 ▼
0.00 ▼		1.00		10.00
▲ 0.00	▲ 0.00	▲ 0.05	▲ 0.01	▲ 0.51
-10.00	-10.00	-10.00	-10.00	-10.00

b) Prefer the close exit (+1), avoiding the cliff (-10)

(1) $\gamma = 0.1$, noise = 0.5

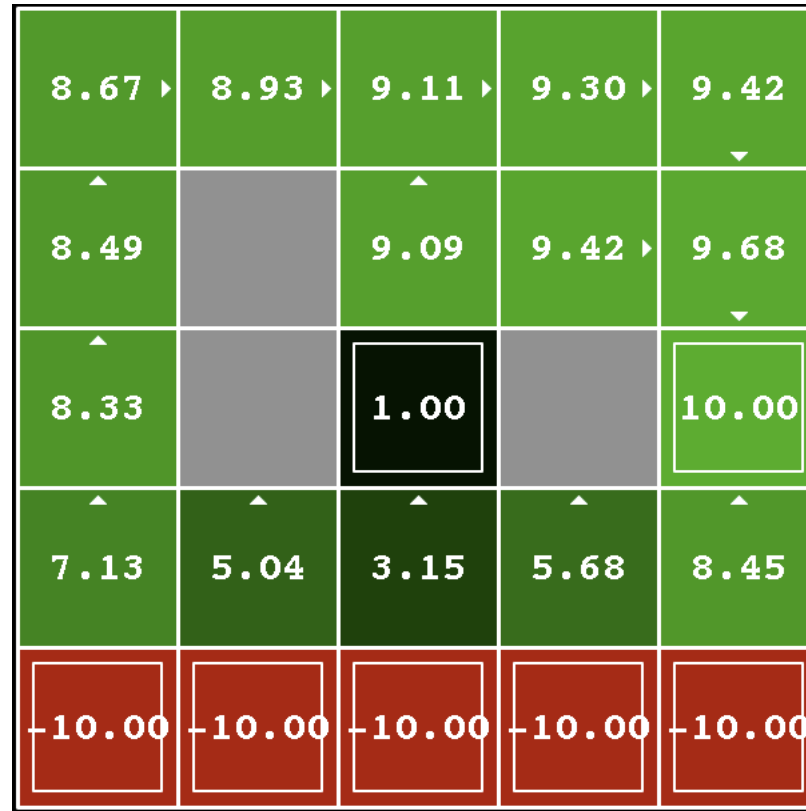
Effect of Discount and Noise



c) Prefer the distant exit (+10), risking the cliff (-10)

(2) $\gamma = 0.99$, noise = 0

Effect of Discount and Noise



d) Prefer the distant exit (+10), avoiding the cliff (-10)

(2) $\gamma = 0.99$, noise = 0.5

Practice: Computing Actions

- Which action should we chose from state s :
 - Given optimal values V ?

$$\arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- Given optimal q-values Q ?

$$\arg \max_a Q^*(s, a)$$

- Lesson: actions are easier to select from Q 's!

Aside: Q-Value Iteration

- Value iteration: find successive approx optimal values
 - Start with $V_0^*(s) = 0$.
 - Given V_i^* , calculate the values for all states for depth $i+1$:

$$V_{i+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_i(s')]$$

- But Q-values are more useful!
 - Start with $Q_0^*(s, a) = 0$
 - Given Q_i^* , calculate the q-values for all q-states for depth $i+1$:

$$Q_{i+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q_i(s', a')]$$

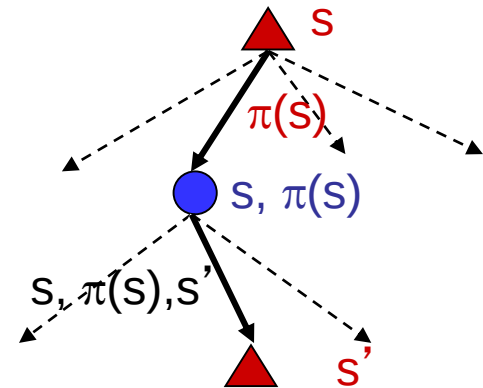
Outline

- MDPs
 - Definition & History
 - MDPs as Search Trees
 - Utility of a Sequence of Actions
- Solving MDPs
 - Bellman Equation
 - Value Iteration
 - Policy Iteration
- Partial Observable MDP (POMDP)

Utilities for Fixed Policies

- Another basic operation: compute the utility of a state s under a fix (general non-optimal) policy
- Define the utility of a state s , under a fixed policy π :
 $V^\pi(s)$ = expected total discounted rewards (return) starting in s and following π
- Recursive relation (one-step look-ahead / Bellman equation):

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')]$$



Policy Evaluation

- How do we calculate the V 's for a fixed policy?
- Idea one: modify Bellman updates

$$V_0^\pi(s) = 0$$

$$V_{i+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_i^\pi(s')]$$

- Idea two: it's just a linear system, solve with Matlab (or whatever)

Policy Iteration

- Problem with value iteration:
 - Considering all actions each iteration is slow: takes $|A|$ times longer than policy evaluation
 - But policy doesn't change each iteration, time wasted
- Alternative to value iteration:
 - **Step 1: Policy evaluation:** calculate utilities for a fixed policy (not optimal utilities!) until convergence (fast)
 - **Step 2: Policy improvement:** update policy using one-step lookahead with resulting converged (but not optimal!) utilities (slow but infrequent)
 - Repeat steps until policy converges
- This is **policy iteration**
 - It's still optimal!
 - Can converge faster under some conditions

Policy Iteration

- **Policy evaluation:** with fixed current policy π , find values with simplified Bellman updates
 - Iterate until values converge

$$V_{i+1}^{\pi_k}(s) \leftarrow \sum_{s'} T(s, \pi_k(s), s') [R(s, \pi_k(s), s') + \gamma V_i^{\pi_k}(s')]$$

- **Policy improvement:** with fixed utilities, find the best action according to one-step look-ahead

$$\pi_{k+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_k}(s')]$$

Comparison

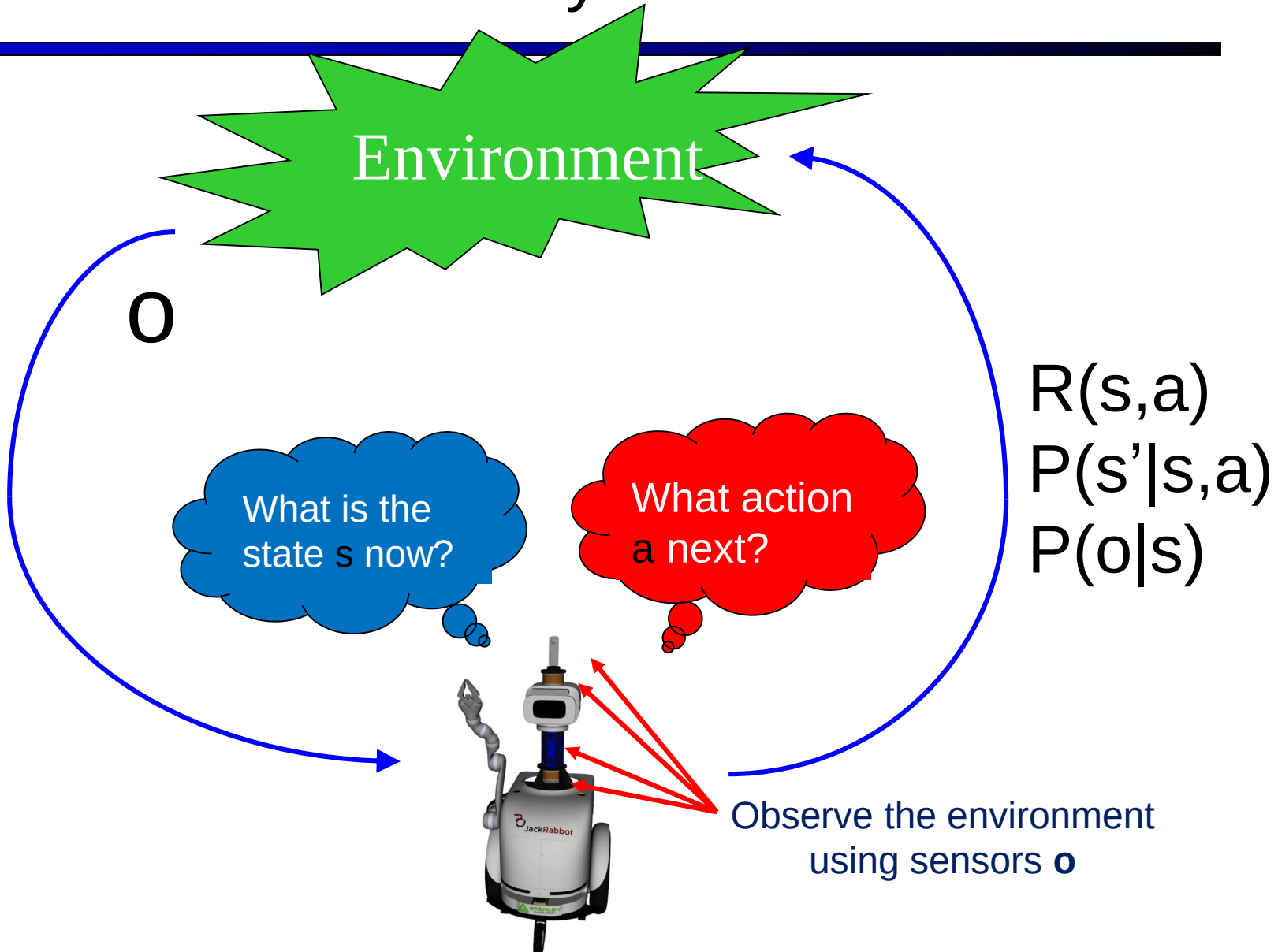
- In value iteration:
 - Every pass (or “backup”) updates both utilities (explicitly, based on current utilities) and policy (possibly implicitly, based on current policy)
- In policy iteration:
 - Several passes to update utilities with frozen policy
 - Occasional passes to update policies
- Hybrid approaches (asynchronous policy iteration):
 - Any sequences of partial updates to either policy entries or utilities will converge if every state is visited infinitely often

Outline

- MDPs
 - Definition & History
 - MDPs as Search Trees
 - Utility of a Sequence of Actions
- Solving MDPs
 - Bellman Equation
 - Value Iteration
 - Policy Iteration
- Partial Observable MDP (POMDP)

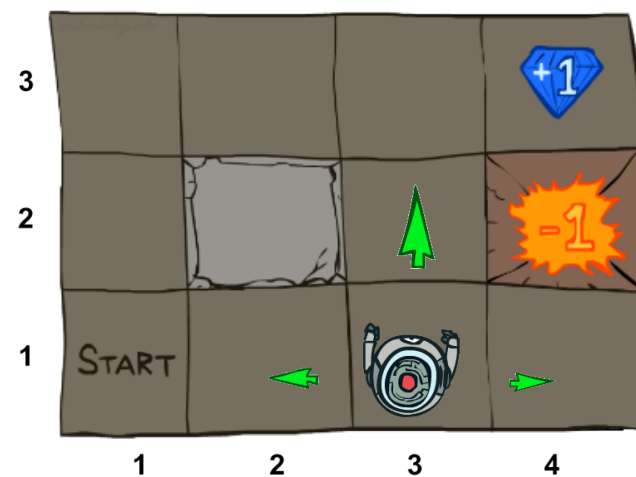
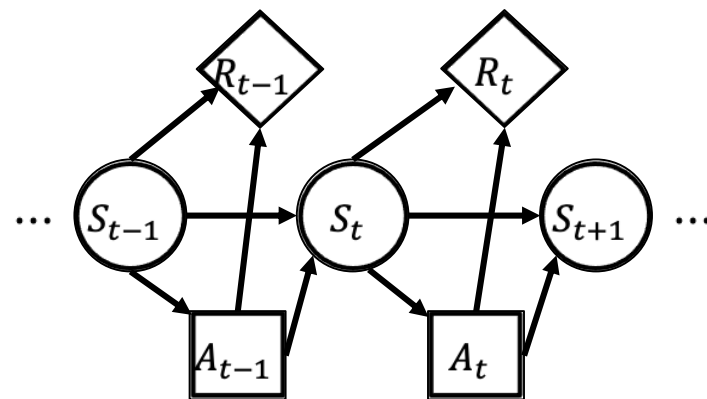
Partial Observable MDP (POMDP)

Plan Under Uncertainty in States & Action



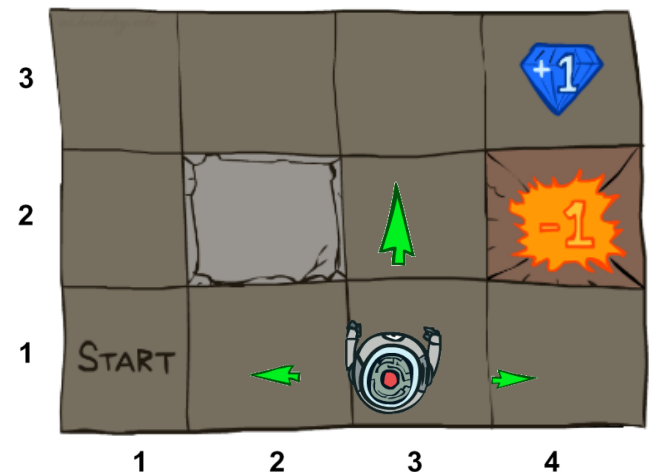
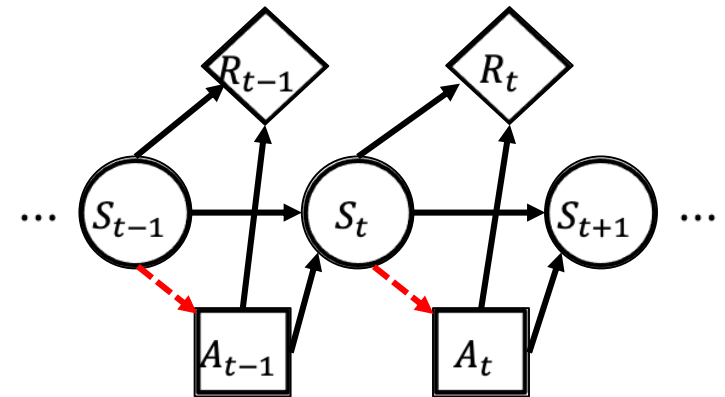
POMDP

- An MDP is defined by
 - Set of states S
 - Set of actions A
 - Transition function $T(s,a,s')$
 - Gives the probability of s' , $P(s' | s, a)$
 - Reward function $R(s,a)$
 - Gives the value the agent receives



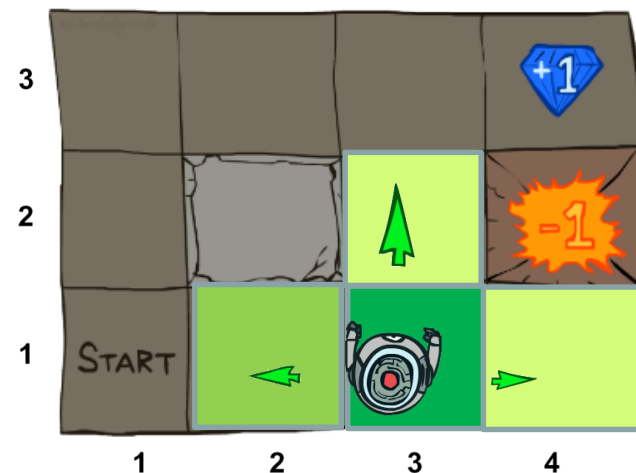
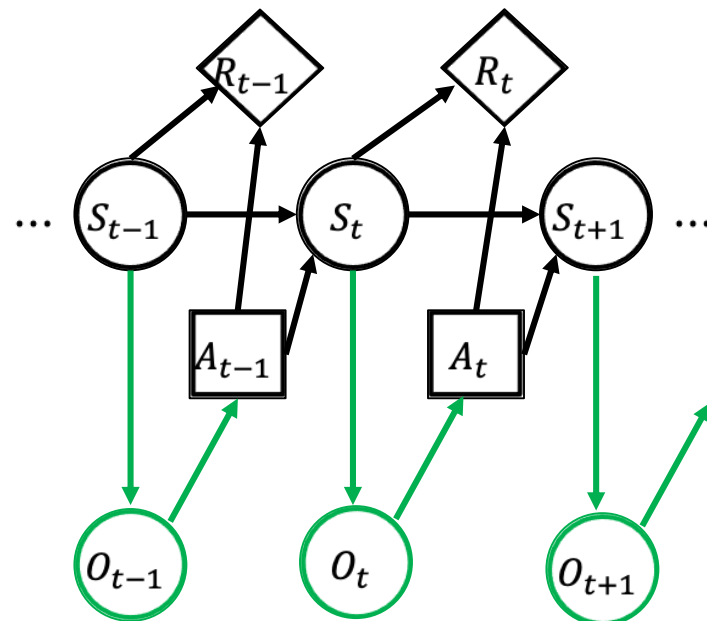
POMDP

- An MDP is defined by
 - Set of states S
 - Set of actions A
 - Transition function $T(s,a,s')$
 - Gives the probability of s' , $P(s' | s, a)$
 - Reward function $R(s,a)$
 - Gives the value the agent receives



POMDP

- A **POMDP** is defined by
 - Set of states S
 - Set of actions A
 - Transition function $T(s,a,s')$
 - Gives the probability of s' , $P(s' | s, a)$
 - Reward function $R(s,a)$
 - Gives the value the agent receives
 - Set of Observation O
 - Likelihood function $G(o,s)$
 - Gives the likelihood of o , $P(o | s)$



Next Week

Reinforcement Learning